Progress Report II

Data Preprocessing Pipeline & Baseline ML Model

By

Muhammad Haider Iqbal 2023416

Muhammad Danyal 2023406

Faculty of Computer Science and Electrical Engineering

Bachelors of Science in Cybersecurity

1. Objective

This report details the successful implementation of the first major component of the Dynamic Threat Response Agent (DTRA), as outlined in our project proposal.

The primary objective for this milestone was to fulfill

Progress Report II:

Data preprocessing pipeline + baseline ML models with performance metrics.

This involved building the "Detector" brain of our agent. The goal was to create a system that can:

1. Process a large-scale, raw cybersecurity dataset (CICIDS2017).
2. Train a baseline Machine Learning model to classify network traffic as "Benign" (Leaf) or "Malicious" (Robber).
3. Evaluate this model using the *exact* metrics specified in our proposal: **precision, recall, f1-score,** and **false positive analysis** (for "Alert Fatigue").
4. Prove the model can output **"confidence probabilities"** (our "Danger Score"), which is a key requirement for the *next* stage of our project (the A* "Decider" agent).

2. Methodology & Implementation

We followed a 5-step process using Google Colab, Pandas, and Scikit-learn.

Step 1: Dataset Acquisition (Proposal Sec. 4)

- **Dataset:** We used the **CICIDS2017** dataset, as specified in our proposal.
- **Preparation:** The 8 separate .csv files (for each day) were programmatically combined into a single, unified dataset of 2.83 million network flows.

Step 2: Data Preprocessing Pipeline

- **Sampling:** From the 2.83 million rows, a **random sample of 100,000 rows** was taken. This is a standard data science technique. It provides a dataset that is large enough to be statistically representative and produce a highly accurate baseline, while also allowing for rapid training (under 1 minute vs. 1+ hours for the full dataset). This was a deliberate choice to optimize for efficient prototyping.

- **Cleaning:**
    1. All column names were stripped of excess spaces.
    2. Non-numeric values (like Infinity) were replaced with NaN to be properly handled.
- **Label Engineering:** The text-based Label column was converted into a binary label column (where 0 = BENIGN and 1 = Attack), aligning with our classification goal.
- **Feature Identification:** We programmatically confirmed our dataset contains **79 numeric features** (e.g., Flow Duration, Destination Port) and no categorical (text) features.

Step 3: Building the Pipeline

We built a simplified, highly efficient Pipeline using Scikit-learn. This "Automatic Cleaner" does two things:

1. **SimpleImputer:** Automatically finds any missing data (NaN) in the 79 "clue" columns and fills them using the median value.
2. **StandardScaler:** "Rescales" all 79 features so they are in a standard range, which improves model accuracy.

Step 4: Baseline Model (Proposal Sec. 7)

- **Model:** We selected **LogisticRegression** as our baseline model.
- **Why?** Our proposal's "AI Approach" demands a "Machine Learning (Detection)" model that provides **"probabilistic confidence."** LogisticRegression is the perfect baseline because it is designed to output a precise probability (our "Danger Score") between 0.0 and 1.0.

Step 5: Training & Evaluation (Proposal Sec. 8)

The 100,000-row sample was split into a 70% "Study Pile" (70,000 rows) and a 30% "Final Exam" pile (30,000 rows). The full pipeline (Cleaner + Model) was trained on the "Study Pile."

3. Results & Performance Metrics

The trained "Detector" was unleashed on the 30,000-row "Final Exam" pile. The results are exceptionally successful and meet all objectives for this report.

A. Objective: "Quantify Alert Confidence"

The model successfully generated "Danger Scores" (probabilities). This proves we can feed this

output to our "Decider" agent.

Output:

```
--- A. Example 'Confidence Scores' (first 5 test samples) ---
  Sample 1: 'Danger Score' = 0.0066 (0.7%)
  Sample 2: 'Danger Score' = 0.0001 (0.0%)
  Sample 3: 'Danger Score' = 0.0000 (0.0%)
  Sample 4: 'Danger Score' = 0.2714 (27.1%)
  Sample 5: 'Danger Score' = 0.0009 (0.1%)
```

Analysis: The model is correctly identifying "Leaf" (Benign) events and assigning them near-zero "Danger Scores," just as required.

B. Objective: "Performance Metrics" (Precision, Recall, F1-Score)

The model's "Report Card" shows a high-performing baseline.

Output:

```
--- B. 'The Report Card' (Classification Report) ---
              precision    recall  f1-score   support

           0       0.96      0.97      0.96     24037
           1       0.86      0.83      0.85      5963

    accuracy                           0.94     30000
   macro avg       0.91      0.90      0.91     30000
weighted avg       0.94      0.94      0.94     30000
```

Analysis:

- **accuracy: 0.94**: Our baseline "Detector" is **94% accurate** overall.
- **recall (for '1'): 0.83**: The model successfully **"catches" 83%** of all real "Robbers" (attacks).
- **precision (for '1'): 0.86**: When the model *does* yell "Robber!", it is **correct 86% of the time.**

C. Objective: "Reduction in false positives" (Alert Fatigue)

The "Mistake Counter" (Confusion Matrix) gives us the *exact* number for our "Alert Fatigue" problem.

Output:

```
--- C. 'The Alert Fatigue' Score (Confusion Matrix) ---
          [Predicted Leaf] [Predicted Robber]
Actual Leaf:    [   23249    ]      [   788      ]  <-- False Positive (Alert Fatigue)!
Actual Robber:  [   992      ]      [   4971     ]  <-- False Negative (Missed Attack)!
```

Analysis:

- **False Positive (Alert Fatigue): 788.**

  This is our key number. Out of 24,037 "Leaves," our system still generated **788 false "BEEPs."** This is the metric we will try to reduce in our final model.

- **False Negative (Missed Attack): 992.**

  The model also *missed* 992 real attacks.

4. Conclusion

This milestone is a complete success. We have:

1. Built a robust, reusable "Data Preprocessing Pipeline" that can clean and prepare the entire *CICIDS2017* dataset.
2. Trained a baseline LogisticRegression "Detector" that is **94% accurate** by strategically sampling our data.
3. Generated all the **"Performance Metrics"** (precision, recall, f1-score) required by our proposal.
4. Successfully generated the **"Confidence Probabilities"** (Danger Scores) that are essential for our next step.
5. Quantified our core problem: our baseline model still produces **788 False Positives**, which is the **"Alert Fatigue"** we aim to solve.